

Manual Técnico do Script abuse.py - Sistema Hórus

Versão do Documento: 1.0

Data de Atualização: Dezembro de 2025

Tecnologia Principal: Python 3 + IMAP (imaplib) + BeautifulSoup + Pandas

Este manual técnico descreve em detalhes o script `abuse.py`, que atua como o módulo de coleta via e-mail para relatórios de vazamentos na conta `abuse@fab.mil.br` no sistema Hórus. O script conecta-se ao servidor IMAP, verifica pastas específicas por e-mails não lidos, extrai corpos (HTML/texto) e anexos, converte planilhas para CSV e salva tudo em pasta dedicada para processamento posterior. Ele garante ingestão automática de dados de fontes externas via e-mail.

O documento explica as funcionalidades principais, configurações, funções disponíveis, restrições de uso e mecanismos de coleta implementados. Ele é destinado a administradores, desenvolvedores e mantenedores que precisam compreender ou modificar o comportamento da ingestão via Abuse/Zimbra.

1. Introdução e Propósito

O `abuse.py` é o componente inicial do fluxo de ingestão via e-mail no Hórus, uma plataforma de Threat Intelligence. Ele acessa a caixa postal `abuse@fab.mil.br`, baixa e-mails não lidos de pastas monitoradas ("HORUS" e "Junk"), extrai texto limpo de corpos HTML, salva anexos (convertendo planilhas para CSV) e armazena em `/Dados/zimbra` para parsing subsequente. O script opera de forma autônoma via execução direta ou integrado a orquestradores (ex: `daily_horus_exec.sh`), com logging para debug.

Ele utiliza `imaplib` para IMAP/TLS, `email` para parsing de mensagens, `BeautifulSoup` para limpeza HTML e `Pandas` para conversão de planilhas. O foco é em coleta robusta, ignorando erros de SSL para compatibilidade e desabilitando warnings de `urllib3`.

1.1 Dependências e Integrações

- **Linguagem:** Python 3.
- **Protocolo:** IMAP (porta 143 com starttls).
- **Bibliotecas:** os (pastas), ssl e imaplib (conexão), email e decode_header (parsing), BeautifulSoup (HTML clean), pandas (Excel/ODS to CSV), re (clean filenames), urllib3 (disable warnings).
- **Integrações Externas:** Servidor IMAP fab.mil.br; pasta /Dados/zimbra para output.
- **Scripts Relacionados:** Chamado por daily_horus_exec.sh (orquestração), output processado por zimbra_parser.py.

2. Funcionalidades Principais

O `abuse.py` fornece coleta via IMAP para relatórios de abuse, com foco em extração e salvamento. Abaixo, as principais funcionalidades:

- **Conexão IMAP:** Estabelece TLS com `cert_none` para ignorar validações, login com credenciais hardcoded.
- **Verificação de Pastas:** Itera sobre `TARGET_FOLDERS` ("HORUS", "Junk"), seleciona pasta, busca UNSEEN (não lidos).
- **Limite de E-mails:** Processa últimos `NUM_EMAILS` (50) para eficiência.
- **Extração de Corpo:** Decoda subject/from, extrai text/plain ou text/html (limpo via BS4), salva como `[ID]_BODY.txt`.
- **Download de Anexos:** Salva attachments com prefixo `[ID]_`, converte .ods/.xls/.xlsx para CSV (concat sheets, add `ORIGEM_ABA`), remove original se sucesso.
- **Logging:** Dual (print + `LOG_FILE`), com mensagens detalhadas por pasta/UID/erros.
- **Encerramento:** Marca como \Seen, logout.

3. Configurações

O script define configurações hardcoded e de ambiente via os:

- **IMAP:** IMAP_SERVER="imap.fab.mil.br", PORT=143, USER="abuse@fab.mil.br", PASS="%CDC@3R!Cyb3r@2025*".
- **Pastas:** TARGET_FOLDERS=["HORUS", "Junk"], NUM_EMAILS=50.
- **Pastas:** BASE_FOLDER=r"/Dados", DOWNLOAD_FOLDER="/Dados/zimbra", LOG_FILE="debug_email_horus.log".
- **SSL:** Context com check_hostname=False, verify_mode=CERT_NONE.
- **Outras:** urllib3 disable_warnings para InsecureRequestWarning.

Restrições: Credenciais hardcoded (risco – mova para .env); paths Windows-like (r"/Dados" – ajuste para Linux se necessário); BS4 para HTML (instale beautifulsoup4).

4. Funções e Métodos Principais

O script define funções modulares para cada etapa. Cada função é descrita abaixo, incluindo parâmetros, funcionalidades e restrições:

- **log(msg):** Print + append a LOG_FILE com utf-8. Sem retorno.
- **clean_folder_name(folder_name):** Substitui não-alphanum por '_'. Retorna str.
- **extract_text_from_html(html_content):** Usa BS4 para remover scripts/styles, get_text com \n. Retorna str clean ou original em erros.
- **convert_spreadsheet_to_csv(filepath):** Lê Excel/ODS com engine apropriado (odf/xlrd/openpyxl), concat sheets com 'ORIGEM_ABA', salva como [file]_CONVERTED.csv (sep=';', utf-8-sig). Retorna path ou None em erros.
- **process_mailbox(mail, folder_name):** Seleciona pasta, busca UNSEEN, processa últimos NUM_EMAILS: fetch RFC822, parse msg, decode subject/from, extrai body (plain/html com clean) ou attachments (salva/convert). Salva body como [ID]_BODY.txt. Marca \Seen.
- **main():** Cria ssl_context, conecta IMAP, starttls, login, processa cada TARGET_FOLDERS via process_mailbox, logout.
- **if name == "main":** Chama main().

5. Restrições e Segurança

- **Pastas:** Foco em "HORUS"/"Junk"; sem suporte a subpastas. Quotes em select para espaços.
- **Segurança:** PASS hardcoded (mova para .env); cert_none/hostname False (risco MITM – use apenas em rede interna). Decode errors="ignore" para robustez.
- **Performance:** Limite 50 e-mails; Pandas para conversão (memória para grandes Excel).
- **Erros:** Logs em exceções; continua em falhas por UID/pasta. Sem retry em login/fetch.
- **Restrições Gerais:** Depende de pip installs (bs4, pandas, openpyxl, odfpy, xlrd); paths hardcoded (ajuste BASE_FOLDER). Sem anexo overwrite check.

Nota Final

O `abuse.py` é crítico para ingestão via e-mail, mas alterações devem preservar logging e conversões para conformidade. Consulte o código para extensões ou adicione filtros (ex: subject keywords). Em produção, monitore LOG_FILE para falhas de conexão e mova credenciais para segredos.

Manual Técnico do Script `zimbra_parser.py` – Sistema Hórus

Versão do Documento: 1.0

Data de Atualização: Dezembro de 2025

Tecnologia Principal: Python 3 + Pandas/NumPy + Heurísticas/ML (classe transplantada)

Este manual técnico descreve em detalhes o script `zimbra_parser.py`, que atua como o módulo de parsing inteligente para arquivos baixados via Abuse/Zimbra no sistema Hórus. O script lista arquivos em `/Dados/zimbra`, extrai credenciais usando heurísticas/ML, insere no banco via processador externo, move para `processed` e aciona tíquete RT se novos. Ele garante processamento automático pós-coleta.

O documento explica as funcionalidades principais, configurações, funções disponíveis, restrições de uso e mecanismos de extração implementados. Ele é destinado a administradores, desenvolvedores e mantenedores que precisam compreender ou modificar o comportamento do parsing de relatórios de vazamentos.

1. Introdução e Propósito

O `zimbra_parser.py` é o componente de extração e processamento para arquivos de Abuse/Zimbra no Hórus, uma plataforma de Threat Intelligence. Ele usa uma classe ML/heuristic (de `credenciais_ml`) para identificar usuário/senha/site em textos desformatados, filtra por domínios alvo, deduplica via hashes e insere no DB. Se novos, chama `create_rt_ticket_abuse.py`. O script opera autônomo via execução direta ou integrado (ex: `daily_horus_exec.sh`), com logging para monitoramento.

Ele utiliza `pandas/numpy` para data handling, `re/urllib` para parsing, `hashlib` para hashes e `subprocess` para RT. O foco é em extração precisa, ignorando warnings para logs limpos.

1.1 Dependências e Integrações

- **Linguagem:** Python 3.
- **Banco de Dados:** Integra com `credenciais_processor` para insert.
- **Bibliotecas:** `os/shutil` (files), `re/sys` (paths/regex), `pandas/numpy` (data/warnings), `hashlib` (SHA1), `datetime` (timestamps), `urllib.parse` (sites), `typing` (anotações), `subprocess` (RT call).
- **Integrações Externas:** Pasta `/Dados/zimbra` (input) e `zimbra_processed` (output); chama `create_rt_ticket_abuse.py`.
- **Scripts Relacionados:** Input de `abuse.py`, output para `credenciais_processor`, chama RT script.

2. Funcionalidades Principais

O `zimbra_parser.py` fornece parsing ML para relatórios Zimbra, com foco em extração/inserção/alert. Abaixo, as principais funcionalidades:

- **Listagem de Arquivos:** Itera files em INPUT_FOLDER, processa cada.
- **Extração ML:** Usa HorusCredentialExtractorML para parsear conteúdo (texto/CSV), extrair/formatar credenciais únicas com hashes.
- **Inserção no Banco:** Chama processar_lista_externa com source_bttnng=False, acumula novos.
- **Movimentação:** Move para PROCESSED_FOLDER, adiciona timestamp se conflito.
- **Alerta RT:** Se novos >0, chama create_rt_ticket_abuse.py via subprocess, loga output/stderr.
- **Logging:** Dual (print + LOG_FILE) com timestamps.
- **Warnings:** Ignora pandas/numpy para logs clean.

3. Configurações

O script define configurações de paths via os/sys:

- **Pastas:** BASE_FOLDER=r"/Dados", INPUT_FOLDER="/Dados/zimbra", PROCESSED_FOLDER="/Dados/zimbra_processed", LOG_FILE="zimbra_parser.log" relativo.
- **Classe ML:** Domínios alvo (.mil.br etc.), patterns regex (email/cpf/url etc.), blacklist (palavras como 'infected').
- **Outras:** Warnings.filterwarnings('ignore') para pandas/numpy.

Restrições: Paths Windows-like (r"/Dados" – ajuste para Linux); depende de credenciais_processor no parent dir.

4. Funções e Métodos Principais

O script define log/main e classe HorusCredentialExtractorML com métodos. Cada um é descrita abaixo, incluindo parâmetros, funcionalidades e restrições:

- **log(msg)**: Print + append a LOG_FILE com timestamp/utf-8.
- **process_file_content(filepath, extractor)**: Lê como str ou pd.read_csv (se .csv), chama extract_credentials. Retorna lista dicts ou [] em erros.
- **HorusCredentialExtractorML.init()**: Inicializa dominios/patterns/regex/blacklist compilados.
- **HorusCredentialExtractorML.extract_credentials(content)**: Se str, split lines; se DF, to_string. Itera lines, aplica regex, valida user (cpf/email/nome), parse site, filtra por dominios/blacklist/hex/hostname/mascarada. Deduplica via hash. Retorna lista dicts.
- **main()**: Cria PROCESSED_FOLDER, lista files, instancia extractor, processa cada (extract/insert/move), acumula novos, chama RT se >0 via subprocess.
- **if name == "main"**: Chama main().

5. Restrições e Segurança

- **Formatos**: Suporta texto/CSV; assume UTF-8. Deduplica por hash usuário+senha+site.
- **Segurança**: Hashes SHA1; filtra por dominios/blacklist. Subprocess para RT (sem check=True – continua em erros).
- **Performance**: Pandas to_string para DF; seen_hashes para dedup. Restrição: Sem paralelismo.
- **Erros**: Logs em exceções; continua em falhas por file. Sem retry em insert/RT.
- **Restrições Gerais**: Depende de pip (pandas/numpy); paths hardcoded. Sem suporte a outros formatos (ex: JSON).

Nota Final

O `zimbra_parser.py` é avançado com ML transplantado, mas alterações devem preservar validações e logging para conformidade. Consulte o código para extensões ou melhore blacklist. Em produção, monitore LOG_FILE para erros de extração e integre com backups de pastas.